Project statement: "Machine and Deep Learning for Multimedia Retrieval"(I-ILIA-014)

1 Introduction

As announced during the session, this project will build on the knowledge acquired during the course (Fig. 1) to develop a multimedia indexing and retrieval application.

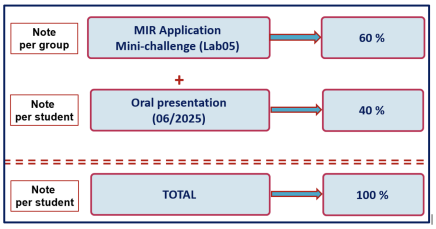

Figure 1: Assessment methods for the AA

Projects will be carried out in groups of two with the following deadlines:

- **Project submission deadline** (report 20 pages, code and user manual): **June 1, 2026?** via Moodle
- **Project presentation format**: in person **(on the day of the exam session)**
- **Project presentation date**: **June 15, 2026?**: Each group will have a maximum of **15 minutes** to present the project, followed by **5 to 10 minutes** of questions.

The presentation schedule will be as follows:

- 08:30 - 09:00 : Group 01 (Lucas Colinet and Gabriel Mroue)
- 09:00 - 09:30 : Group 02 (Manon Rustin and Margaux Leclercq)
- 09:30 - 10:00 : Group 03 (Jérémy Delnatte and Tonesko Djoufo Tafejo)
- 10:00 - 10:30 : Group 04 (Radu Florea and Gilles Jaunart)

********** **BREAK 1** **********

- 11:10 - 11:40 : Group 05 (Marie Canfyn and Flavio Drogo)
- 10:40 - 11:10 : Group 06 (Lucas Bricoult and Alex Thayse)
- 11:40 - 12:10 : Group 07 (Sacha De Mulder and Roy Ebwelle)
- 12:10 - 12:45 : Group 08 (Paul Toussaint, Julien Xu et Alla Abdelouahad)

2 Part I: Unimodal Image Search Engine with two levels

The objective of this part is to develop an image retrieval engine using feature descriptors following these steps:

1. Index the database using the descriptors of your choice. If multiple descriptors are selected, the system must allow their combination **(see Assignment TP3.3: strongly recommended)**
2. Implement the search process by allowing the user to choose the similarity measure (Euclidean distance, Correlation, Chi-square, Bhattacharyya, Brute Force Matcher, FLANN, etc.).
3. Display the Top-20 and Top-50 retrieved images for each query image.
4. Compute evaluation metrics : Recall (R), Precision (P), Average Precision (AP), Mean Average Precision (MAP), R-Precision.

You may choose to work either in Python or C++. However, this choice must be considered in the optional part of the project, which consists of deploying your application on a cloud resource.

Odd-numbered groups **1, 3, 5 and 7** will work on the **"Cars"** database containing 10 classes (10,000 images). To test your engine, these queries must be performed. **DB link:** `https://nextcloud.ig.umons.ac.be/s/BooirG6KkHJ58XB`

Query Index	Class	Images
R1, R2, R3	0	0_1_BMW_X3_207, 0_0_BMW_Serie3Berline_74, 0_2_BMW_i8_299
R4, R5, R6	2	2_0_Volkswagen_Touareg_2822, 2_4_Volkswagen_Polo_3463, 2_9_Volkswagen_T-Roc_4209
R7, R8, R9	4	4_2_Opel_vivarofourgon_5999, 4_4_Opel_Insignatourer_6353, 4_9_Opel_zafiralife_6887
R10, R11, R12	6	6_0_Hyundai_Nexo_8282, 6_3_Hyundai_i10_8837, 6_5_Hyundai_i30_9125
R13, R14, R15	8	8_1_Ford_Puma_11276, 8_5_Ford_Explorer_11897, 8_6_Ford_Focus_11951

Table 1: Query and Image Classification - Odd-numbered groups (1, 3, 5, 7, and 9)

Even-numbered groups **(2, 4, 6 and 8)** will work on the same **"Cars"** database. To test the engine, the following queries must be performed (3 queries per class):

Query Index	Class	Images
R1, R2, R3	1	1_4_Kia_stinger_1990, 1_2_Kia_sorento_1675, 1_9_Kia_stonic_2677
R4, R5, R6	3	3_1_Renault_Twingo_4487, 3_0_Renault_grandscenic_4372, 3_5_Renault_clio_5101
R7, R8, R9	5	5_0_Mercedes_ClasseCLS_7059, 5_4_Mercedes_GLEcoupe_7428, 5_8_Mercedes_CLA_7992
R10, R11, R12	7	7_0_Peugeot_508break_9591, 7_3_Peugeot_Rifter_10091, 7_6_Peugeot_3008_10530
R13, R14, R15	9	9_0_Audi_A6_12268, 9_3_Audi_Q7_12722, 9_4_Audi_A1_12910

Table 2: Query and Image Classification - Even-numbered groups (2, 4, 6, 8 and 10)

The results of the descriptor calculations for 'indexing' must be presented according to Table 1 below :

Your best descriptors	Descr. name(s)	Indexing Time (s)	Descr. size (MB)	Av. Search Time/image (s)
Descriptor No. 01				
Descriptor No. 02				
Descriptor No. 03				

Table 3: Performance metrics of the best descriptors

The expected results for each query must be presented as follows (You may also add a 'TopMax' column (Max: number of images in the relevant class):

Query	R (Recall)		P (Precision)		AP		mAP	
Index	Top50	Top100	Top50	Top100	Top50	Top100	Top50	Top100
R1								
R2								
...								
R15								

Table 4: Search engine precision metrics

Note 1

It is mandatory to select at least two deep learning descriptors (one CNN and one Vit) among your 3 best descriptors. Due to its computational complexity, you may reduce the image resolution when calculating SIFT descriptors.

3 Part II : "Multimodal Search Engine"

In Part 2, we propose developing a multimodal search engine using the "Flickr8K" database (**8,000 images, where each image is associated with 5 text descriptions**). You can leverage a model pre-trained on image-text pairs, such as **CLIP (Contrastive Language-Image Pretraining)** developed by OpenAI. This type of model is designed to learn a joint representation between images and text descriptions. The advantages of this approach are as follows:

- **Joint Representation:** A shared vector space is used for both images and text, which improves *cross-modal retrieval* (finding an image from text or text from an image).
- **Natural Alignment of Modalities:** Models like CLIP are trained on large-scale datasets of images and descriptions, allowing for better semantic matching between text and image.
- **Improved Generalization:** Unlike approaches using separate models, a multimodal model can better adapt to new complex queries.

The following steps should be followed to complete this part:

1. **Dataset Preparation:** Download the Flickr8k multimodal dataset:`https://github.com/goodwillyoga/Flickr8k_dataset`
2. **Utilization of a Contrastive Multimodal Model:** Index images and text using the CLIP architecture.

- **Image Indexing:** Convert each image in the database into an embedding vector using the CLIP image encoder.
- **Description Indexing:** Pass each text description through the CLIP text encoder to obtain its corresponding vector.

3. **Storage and Indexing:**
   - Build an embedding database containing the representations of both images and texts.
   - Implement your engine using **FAISS** (Facebook AI Similarity Search) for efficient fast k-nearest neighbors search.
4. **Retrieval and Evaluation:**
   - Select a corpus of 3 different images and 3 different texts to evaluate your search engine.
   - Implement a *text-to-image* query to find the most similar images in the latent space.
   - Test *image-to-text* retrieval (inverse search) to find descriptions matching a given image.
   - Evaluate the quality of the results using metrics such as **Precision, Recall, and mAP** (mean Average Precision).
5. **Optional:** Test and evaluate your engine with another contrastive architecture of your choice (e.g., **AlignVLM, BLIP, OpenCLIP**, etc.).
6. **Optional:** Present a comparative analysis ofresults between the CLIP-based search engine and your second contrastive VLM.
7. **Optional:** Propose your own improvements to the search engine based on your results analysis.

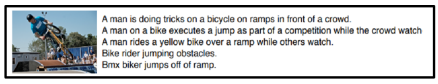

Figure 2: Example data from Flickr8K DB. Source: `https://github.com/goodwillyoga/Flickr8k_dataset`

4 Part III : Cloud Resource Hosting (Optional)

The objective of this part is to host your multimedia search application (from Part 1 and/or 2) on a Cloud resource to provide a service in the form of **Software As A Service (SaaS)**. We suggest following these steps:

1. **Local Indexing (Feature Extraction):** Select your best model(s) and image (and text, if applicable) feature files. Copy them to your virtual machine or deployment resource. *Note: The indexing phase should not be hosted on the deployment resource itself.*
2. **Cloud Resource Configuration/Testing:** Install/configure your deployment environment to test the application **(Part 1 and/or 2)**.
3. **Docker Image Generation:** Create a `Dockerfile` containing necessary instructions to run your application. Your image must handle:
   - **Input:** A query image (or text for Part 2).
   - **Output:** Indices of the most similar images (or texts) and the Recall/Precision curve.
4. **Web Interface Development:** Develop a web page (using Flask, Django, or PHP) to facilitate access to the SaaS service, including:
   - Developer information and a description of the application's features.
   - Interactive elements (buttons, labels, etc.) to launch the search.
   - Display of search results: similar images/texts (with similarity scores) and R/P curves.
5. **Access Configuration:** Configure access to your service using your IP address and a port number of your choice.
6. **Website Customization:** Personalize your site according to your imagination, including a login page.
7. **Dimensionality Reduction:** Reduce the size of your descriptors without compromising the search engine's precision.
8. **Cybersecurity:** Analyze your website's security and implement improvements accordingly.

Note 2

For **parts 1 & 2**, you can use your own PCs, Google Colab, or request access to IG cluster (one access per group).

Note 3

For **part 3**, you may deploy the search engine on a local machine of your choice or on a Cloud VM. If you rent a VM, you can request a reimbursement via an expense claim (up to 25 euros per group). You may also take advantage of free cloud services such as :the Oracle Free Tier, Railway, Render, Pythonanywhere.

Annex

You can watch this video to get a simple and clear idea of the expected work. The image search application uses a Docker image and a web page developed using PHP and HTML.

- **Example of hosting a Python application ("Image Classification with Deep Learning") using Docker and PHP:** link.
- **Example of hosting a C++ image processing application using Docker and PHP:** see this link.